

# BÀI THỰC HÀNH CHƯƠNG 5

## Lập trình CSDL với Spring Data JPA (Nâng cao)

- **Học phần:** Lập trình ứng dụng với Java (14113014)
- **Chương:** Chương 5 - Lập trình CSDL với Spring Data JPA (Nâng cao)
- **Thời lượng:** 3 tiết thực hành

### 1. Mục tiêu bài học

Sau khi hoàn thành bài thực hành này, sinh viên sẽ có khả năng:

**1.1. Kiến thức:** Hiểu rõ vai trò của JpaRepository trong hệ thống phân cấp Repository của Spring Data.

- Nắm vững cơ chế phân trang (Pageable, PageRequest) và sắp xếp (Sort) để tối ưu hóa truy vấn dữ liệu.
- Phân biệt cơ chế truy vấn tự sinh (Derived Query Methods) và truy vấn tự định nghĩa bằng @Query (JPQL và Native SQL).
- Hiểu cách quản lý các mối quan hệ bảng: Một-Một (@OneToOne), Nhiều-Một (@ManyToOne), Một-Nhiều (@OneToMany), Nhiều-Nhiều (@ManyToMany).
- Phân biệt cơ chế tải dữ liệu LAZY vs EAGER và lan truyền thao tác CascadeType.

**1.2. Kỹ năng:** Kế thừa và phát triển dự án từ bài thực hành Chương 4, nâng cấp thực thể và các repository hiện có.

- Định nghĩa các truy vấn nâng cao và các thao tác thay đổi dữ liệu bằng @Modifying + @Query.
- Cấu hình chuẩn xác các mối quan hệ thực thể hai chiều (Bidirectional) sử dụng thuộc tính mappedBy, @JoinColumn, @JoinTable.
- Phát triển các REST API Endpoint hỗ trợ tìm kiếm, lọc, phân trang, sắp xếp và quản lý quan hệ bảng.
- Kiểm thử và đánh giá kết quả API qua Postman/cURL, đồng thời phân tích các câu lệnh SQL được Hibernate sinh ra để tối ưu hóa hiệu năng.

**1.3. Thái độ:** Nhận thức được tầm quan trọng của việc tối ưu hiệu năng CSDL (tránh N+1 query) và tính an toàn giao dịch (Transactions) khi phát triển các hệ thống Enterprise thực tế.

### 2. Chuẩn bị môi trường

Để thực hiện bài thực hành này, sinh viên cần:

- Sử dụng lại dự án ch4-lab-spring-jpa đã hoàn thành ở Chương 4. Có thể tạo một bản sao thư mục dự án và đổi tên thành ch5-lab-spring-jpa-advanced để thực hành.
- MySQL Server đang hoạt động với CSDL gdu\_jpa\_lab đã được tạo từ bài thực hành trước.
- IntelliJ IDEA, Postman hoặc cURL.

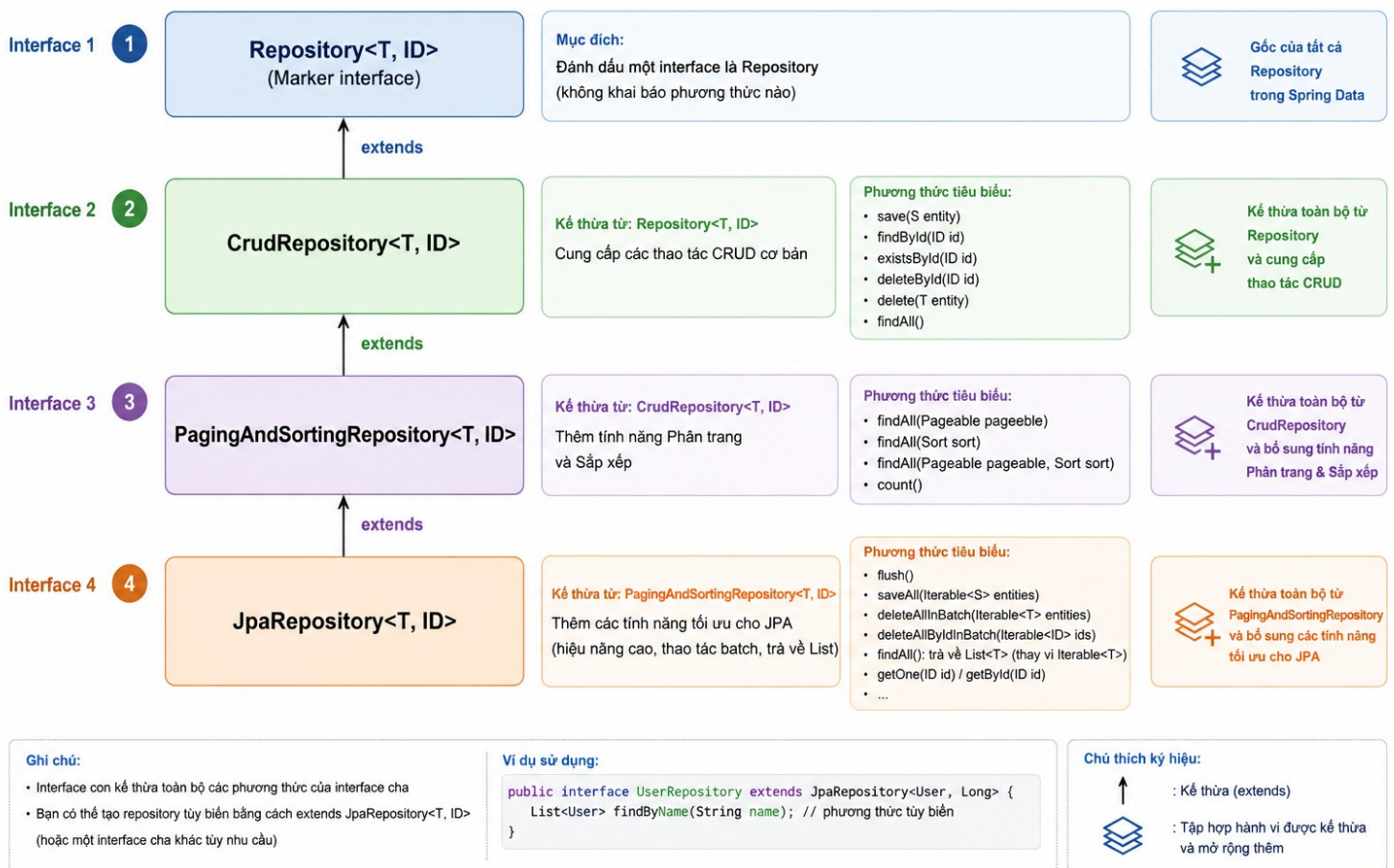
## 3. Kiến trúc nền tảng

### 3.1. Phân cấp Repository

Spring Data JPA tổ chức phân cấp các interface nhằm tối ưu hóa việc tái sử dụng mã nguồn. Interface con kế thừa toàn bộ phương thức của cha và bổ sung các hành vi nâng cao:

- **CrudRepository**: Cung cấp các thao tác CRUD cơ bản.
- **PagingAndSortingRepository**:  
Bổ sung phương thức phân trang và sắp xếp: `findAll(Pageable)` và `findAll(Sort)`.
- **JpaRepository**: Kế thừa từ cả hai interface trên, bổ sung các phương thức đặc thù của JPA như `flush()`, `saveAndFlush()`, `deleteInBatch()`, và thay đổi kiểu trả về của `findAll()` từ `Iterable` thành `List`.

### KIẾN TRÚC PHÂN CẤP REPOSITORY TRONG SPRING DATA JPA



## Các phương thức CRUD có sẵn của CrudRepository

Khi một Repository kế thừa từ CrudRepository hoặc JpaRepository, lập trình viên sẽ mặc nhiên sở hữu các phương thức thao tác dữ liệu tiêu chuẩn sau:

| Phương thức                   | Mô tả hành vi                                                                                                |
|-------------------------------|--------------------------------------------------------------------------------------------------------------|
| save(S entity)                | Lưu trữ thực thể mới hoặc cập nhật thực thể nếu đã tồn tại khóa chính.                                       |
| saveAll(Iterable<S> entities) | Lưu trữ hoặc cập nhật danh sách nhiều thực thể cùng một lúc.                                                 |
| findById(ID id)               | Tìm kiếm thực thể theo khóa chính. Kết quả trả về dưới dạng Optional<T> nhằm tránh lỗi NullPointerException. |
| existsById(ID id)             | Kiểm tra sự tồn tại của thực thể có khóa chính được chỉ định.                                                |
| findAll()                     | Lấy toàn bộ các thực thể hiện có trong bảng CSDL.                                                            |
| findAllById(Iterable<ID> ids) | Lấy danh sách các thực thể tương ứng với danh sách khóa chính truyền vào.                                    |
| count()                       | Đếm tổng số bản ghi hiện có trong bảng.                                                                      |
| deleteById(ID id)             | Xóa bản ghi khỏi CSDL dựa trên khóa chính.                                                                   |
| delete(T entity)              | Xóa bản ghi tương ứng với thực thể truyền vào.                                                               |
| deleteAll()                   | Xóa toàn bộ các bản ghi trong bảng CSDL.                                                                     |

Ví dụ sử dụng repository trong một lớp Service thực tế:

```
@Service
public class ProductService {
    private final ProductRepository productRepository;

    // Sử dụng Constructor Injection để tiêm bean Repository tự sinh bởi Spring
    public ProductService(ProductRepository productRepository) {
        this.productRepository = productRepository;
    }

    public Product saveProduct(Product product) {
        return productRepository.save(product);
    }

    public Optional<Product> findProductById(Long id) {
        return productRepository.findById(id);
    }

    public List<Product> getAllProducts() {
        return productRepository.findAll();
    }
}
```

## Các phương thức nâng cao của JpaRepository

Bên cạnh các phương thức CRUD cơ bản, JpaRepository cung cấp các phương thức nâng cao tối ưu hóa riêng cho cơ chế hoạt động của JPA (Jakarta Persistence API):

| Phương thức nâng cao                   | Ý nghĩa và mục đích sử dụng                                                                                                                                 |
|----------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------|
| flush()                                | Đồng bộ hóa ngay lập tức tất cả các thay đổi đang nằm trong Persistence Context (bộ nhớ đệm) xuống cơ sở dữ liệu.                                           |
| saveAndFlush(S entity)                 | Lưu thực thể và đồng thời thực hiện flush ngay lập tức, không cần đợi transaction hoàn thành.                                                               |
| deleteAllInBatch()                     | Xóa toàn bộ các bản ghi bằng cách sử dụng một câu lệnh SQL DELETE duy nhất thay vì lặp qua từng thực thể (tối ưu hiệu năng vượt trội).                      |
| deleteAllByIdInBatch(Iterable<ID> ids) | Xóa danh sách thực thể theo ID bằng một câu lệnh SQL duy nhất chứa mệnh đề IN.                                                                              |
| getReferenceById(ID id)                | Trả về một proxy reference của thực thể (lazy loading). Rất hữu ích khi chỉ cần thiết lập khóa ngoại mà không muốn tốn chi phí truy vấn SELECT thực thể đó. |

## 3.2. Phân trang (Pagination) và Sorting (Sắp xếp)

Khi phân trang, Spring Data JPA sử dụng ba thành phần chính:

- **Pageable:** Interface mô tả yêu cầu phân trang (trang hiện tại, kích thước trang, sắp xếp)
- **PageRequest:** Lớp triển khai của Pageable, dùng để khởi tạo thông tin phân trang (ví dụ: PageRequest.of(page, size))
- **Page<T>:** Đối tượng chứa danh sách kết quả của trang hiện tại kèm các thông tin metadata (tổng số trang, tổng số bản ghi, có trang tiếp theo không...)

Ví dụ minh họa:

```
// 1. Chỉ thực hiện sắp xếp (Sắp xếp theo trường "price" tăng dần)
List<Product> productsSorted = productRepository.findAll(Sort.by("price").ascending());

// 2. Thực hiện phân trang và sắp xếp đồng thời
// Lấy trang thứ 0 (trang đầu tiên), kích thước trang là 10 bản ghi, sắp xếp theo "price" giảm dần
Pageable pageable = PageRequest.of(0, 10, Sort.by("price").descending());
Page<Product> productPage = productRepository.findAll(pageable);

// 3. Khai thác dữ liệu từ đối tượng Page trả về
int totalPages = productPage.getTotalPages(); // Tổng số trang
long totalElements = productPage.getTotalElements(); // Tổng số bản ghi dưới DB
List<Product> content = productPage.getContent(); // Danh sách dữ liệu của trang hiện tại
```

### 3.3. Derived Query vs @Query

#### 3.3.1. Derived Query

Tiện lợi cho các câu truy vấn đơn giản. Spring Data tự động biên dịch tên phương thức sang SQL dựa vào cú pháp:

**<Verb><Subject>By<Predicate>**

**Verb (Động từ):** Thường bắt đầu bằng find, read, get, query (SELECT); count (COUNT); exists (EXISTS) hoặc delete, remove (DELETE).

**Subject (Chủ thể):** Tùy chọn, thường là tên của Entity (ví dụ: findProductsBy...), Spring Data sẽ bỏ qua phần này khi phân tích cú pháp.

**Predicate (Vị ngữ):** Các điều kiện lọc dựa trên tên các thuộc tính của Entity liên kết bằng các từ khóa logic như And, Or.

Ví dụ khai báo các phương thức trong Repository:

```
public interface OrderRepository extends JpaRepository<Order, Long> {  
    // SELECT * FROM orders WHERE delivery_zip = ?  
    List<Order> findByDeliveryZip(String deliveryZip);  
  
    // SELECT * FROM orders WHERE delivery_zip = ? AND placed_at BETWEEN ? AND ?  
    List<Order> readOrdersByDeliveryZipAndPlacedAtBetween(  
        String deliveryZip, Date startDate, Date endDate);  
}
```

#### Các từ khóa hỗ trợ trong Derived Query

Dưới đây là bảng tổng hợp các từ khóa so sánh logic thường dùng nhất giúp xây dựng mệnh đề WHERE phong phú:

| Từ khóa       | Ví dụ phương thức                          | Ý nghĩa / SQL tương ứng                   |
|---------------|--------------------------------------------|-------------------------------------------|
| And           | findByNameAndPrice(String n, double p)     | WHERE name = ? AND price = ?              |
| Or            | findByNameOrPrice(String n, double p)      | WHERE name = ? OR price = ?               |
| Between       | findByPriceBetween(double min, double max) | WHERE price BETWEEN ? AND ?               |
| LessThan      | findByPriceLessThan(double limit)          | WHERE price < ?                           |
| GreaterThan   | findByPriceGreaterThan(double limit)       | WHERE price > ?                           |
| IsNull / Null | findByDescriptionIsNull()                  | WHERE description IS NULL                 |
| Like          | findByNameLike(String pattern)             | WHERE name LIKE ? (Cần truyền tay %)      |
| Containing    | findByNameContaining(String keyword)       | WHERE name LIKE '%?%' (Tự thêm % hai đầu) |

|              |                                        |                                             |
|--------------|----------------------------------------|---------------------------------------------|
| StartingWith | findByNameStartingWith(String prefix)  | WHERE name LIKE '?%' (Tự thêm % phía sau)   |
| EndingWith   | findByNameEndingWith(String suffix)    | WHERE name LIKE '%?' (Tự thêm % phía trước) |
| OrderBy      | findByNameOrderByPriceAsc(String name) | ORDER BY price ASC                          |
| Not          | findByNameNot(String name)             | WHERE name <> ?                             |
| In           | findByNameIn(Collection<String> names) | WHERE name IN (?, ?, ...)                   |
| True / False | findByActiveTrue()                     | WHERE active = true                         |
| IgnoreCase   | findByNameIgnoreCase(String name)      | WHERE UPPER(name) = UPPER(?)                |
| Top / First  | findTop5ByOrderByPriceDesc()           | SELECT ... LIMIT 5 (Lấy 5 dòng đầu)         |
| Distinct     | findDistinctByName(String name)        | SELECT DISTINCT ...                         |

### 3.3.2. @Query (JPQL)

Viết câu truy vấn hướng đối tượng bằng thực thể Java (Entity) và thuộc tính (Field). Giúp quản lý tên phương thức ngắn gọn và an toàn khi refactor.

```
public interface ProductRepository extends JpaRepository<Product, Long> {

    // Truy vấn thực thể Product hướng đối tượng (sử dụng tên Class và tên thuộc tính)
    @Query("SELECT p FROM Product p WHERE p.price > :minPrice")
    List<Product> findExpensiveProducts(@Param("minPrice") double minPrice);

    @Query("SELECT p FROM Product p WHERE p.name LIKE %:keyword%")
    List<Product> searchByName(@Param("keyword") String keyword);

    @Query("SELECT p FROM Product p WHERE p.category.name = :categoryName")
    List<Product> findByCategory(@Param("categoryName") String categoryName);
}
```

Ưu điểm của việc sử dụng @Query:

- Tên phương thức có thể tự do đặt theo ý muốn (ví dụ: getExpensiveProducts thay vì findByNameGreaterThan), làm mã nguồn trực quan hơn.
- Tránh các lỗi runtime không mong muốn khi thực hiện refactor tên thuộc tính trong Entity mà quên cập nhật tên phương thức repository tương ứng.
- Hiệu năng khởi động ứng dụng tốt hơn do Spring không cần phân tích cú pháp chuỗi ký tự tên phương thức.

### 3.3.3. @Query (Native SQL)

Viết câu truy vấn SQL thực tế liên kết trực tiếp với bảng và cột trong DB. Dùng khi cần gọi hàm hoặc cú pháp đặc thù của hệ quản trị CSDL.

Lưu ý thiết lập thuộc tính **nativeQuery = true**:

```
@Query(value = "SELECT * FROM products WHERE price > :minPrice ORDER BY price DESC  
LIMIT :limit", nativeQuery = true)  
List<Product> findTopExpensiveProducts(@Param("minPrice") double minPrice,  
                                       @Param("limit") int limit);
```

*Lưu ý: Việc lạm dụng Native Query sẽ làm giảm tính độc lập của mã nguồn đối với hệ quản trị CSDL. Nếu dự án chuyển từ MySQL sang Oracle, một số câu Native Query có thể sẽ bị lỗi cú pháp.*

#### 3.3.4. @Modifying

Bắt buộc sử dụng kèm với @Query và @Transactional cho các câu lệnh UPDATE hoặc DELETE thủ công.

Ví dụ minh họa:

```
@Modifying  
@Transactional // Bắt buộc phải có để quản lý giao dịch ghi dữ liệu  
@Query("UPDATE Product p SET p.price = :newPrice WHERE p.id = :id")  
int updatePrice(@Param("id") Long id, @Param("newPrice") double newPrice);  
  
@Modifying  
@Transactional  
@Query("DELETE FROM Product p WHERE p.active = false")  
int deleteInactiveProducts();
```

*Lưu ý: Các phương thức này thường trả về kiểu dữ liệu int biểu thị số lượng dòng bị tác động dưới database, hoặc kiểu void.*

### 3.3. JPA Relationships & Fetch Strategies

- **Owning Side vs Inverse Side:**

Trong quan hệ hai chiều (Bidirectional), phía chứa cột khóa ngoại (Foreign Key) luôn luôn là Owning Side. Phía đối diện là Inverse Side bắt buộc phải khai báo thuộc tính mappedBy chỉ định tên thuộc tính đại diện ở Owning Side.

- **LAZY vs EAGER:**

ManyToOne và OneToOne mặc định là EAGER (Tải ngay lập tức).

OneToMany và ManyToMany mặc định là LAZY (Chỉ tải khi gọi hàm lấy dữ liệu).

**HIỆU NĂNG HỆ THỐNG:** Luôn cấu hình FetchType.LAZY cho mọi quan hệ. Tránh sử dụng EAGER vì rất dễ dẫn tới lỗi hiệu năng kinh điển 'N+1 Query Problem' (CSDL thực hiện 1 câu



*SELECT lấy cha và N câu SELECT phụ lấy từng con).*

## 4. Nội dung thực hành

### Bài 1: Cấu hình Phân trang và Sắp xếp (Pagination & Sorting)

Mục tiêu: Cập nhật REST API hiện có của thực thể Product để hỗ trợ phân trang và sắp xếp danh sách sản phẩm theo yêu cầu của client thông qua các tham số trên URL.

#### Bước 1: Giữ nguyên và cập nhật Entity Product

Entity Product từ Chương 4 đã có các thuộc tính: id, name (cột product\_name), price, sku, createdAt. Chúng ta giữ nguyên các thuộc tính này và sẽ bổ sung quan hệ Nhiều-Một ở Bài 3.

#### Bước 2: Viết phương thức phân trang trong ProductController

Mở tệp ProductController.java trong package vn.edu.gdu.springjpalab.controller. Giữ nguyên các API cũ từ Chương 4 và thêm API lấy sản phẩm phân trang sau:

```
// Lưu ý: Import bổ sung vào đầu file
import org.springframework.data.domain.Page;
import org.springframework.data.domain.Pageable;
import org.springframework.data.domain.PageRequest;
import org.springframework.data.domain.Sort;

// Thêm phương thức này vào class ProductController
// — API Lấy sản phẩm có Phân trang và Sắp xếp (Chương 5) —
@GetMapping("/page")
public ResponseEntity<Page<Product>> getProductsPageable(
    @RequestParam(defaultValue = "0") int page,
    @RequestParam(defaultValue = "10") int size,
    @RequestParam(defaultValue = "price") String sortBy,
    @RequestParam(defaultValue = "asc") String sortDir
) {
    Sort sort = sortDir.equalsIgnoreCase("asc") ?
        Sort.by(sortBy).ascending() :
        Sort.by(sortBy).descending();

    Pageable pageable = PageRequest.of(page, size, sort);

    return ResponseEntity.ok(productRepository.findAll(pageable));
}
```

Ghi chú: *productRepository* kế thừa từ *JpaRepository*, vốn đã gián tiếp kế thừa *PagingAndSortingRepository*, nên phương thức *findAll(Pageable)* đã được tích hợp sẵn mà không cần khai báo thêm.



### Bước 3: Kiểm thử phân trang bằng Postman/cURL

Khởi động ứng dụng Spring Boot và gửi yêu cầu kiểm thử:

*Yêu cầu: Lấy trang đầu tiên (trang 0), kích thước trang là 2 sản phẩm, sắp xếp theo giá giảm dần.*

*Postman: GET http://localhost:8080/api/products/page?page=0&size=2&sortBy=price&sortDir=desc*

Sử dụng cURL:

```
curl "http://localhost:8080/api/products/page?
page=0&size=2&sortBy=price&sortDir=desc"
```

Quan sát cấu trúc JSON trả về. Bên cạnh thuộc tính content chứa danh sách sản phẩm, sẽ thấy thông tin metadata tương ứng (totalPages, totalElements, size, number...).

## Bài 2: Định nghĩa Derived Query Methods và @Query tùy chỉnh

Mục tiêu: Bổ sung các phương thức truy vấn tùy biến vào ProductRepository để thực hiện tìm kiếm sản phẩm theo tên, theo khoảng giá, truy vấn theo danh mục và cập nhật giá hàng loạt.

### Bước 1: Cập nhật ProductRepository

Mở tệp ProductRepository.java trong package vn.edu.gdu.springjpalab.repository. Bổ sung các phương thức truy vấn sau:

```
package vn.edu.gdu.springjpalab.repository;

import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.data.jpa.repository.Modifying;
import org.springframework.data.jpa.repository.Query;
import org.springframework.data.repository.query.Param;
import org.springframework.stereotype.Repository;
import org.springframework.transaction.annotation.Transactional;
import vn.edu.gdu.springjpalab.entity.Product;

import java.math.BigDecimal;
import java.util.List;

@Repository
public interface ProductRepository extends JpaRepository<Product, Long> {

    // 1. Derived Query Method: Tìm kiếm theo tên sản phẩm (không phân biệt hoa thường)
    List<Product> findByNameContainingIgnoreCase(String keyword);

    // 2. Derived Query Method: Tìm sản phẩm có giá nằm trong khoảng [min, max]
    List<Product> findByPriceBetween(BigDecimal minPrice, BigDecimal maxPrice);

    // 3. Custom Query (JPQL): Tìm sản phẩm có giá lớn hơn giá trị tối thiểu
    @Query("SELECT p FROM Product p WHERE p.price > :minPrice")
    List<Product> findExpensiveProducts(@Param("minPrice") BigDecimal minPrice);

    // 4. Custom Query (Native SQL): Lấy 3 sản phẩm có giá cao nhất
    @Query(value = "SELECT * FROM products ORDER BY price DESC LIMIT 3", nativeQuery = true)
    List<Product> findTop3ExpensiveProducts();

    // 5. Modifying & Custom Query: Cập nhật tăng giá sản phẩm hàng loạt theo tỷ lệ
```

```

    @Modifying
    @Transactional
    @Query("UPDATE Product p SET p.price = p.price * :rate WHERE p.id = :id")
    int updateProductPrice(@Param("id") Long id, @Param("rate") BigDecimal rate);
}

```

## Bước 2: Bổ sung REST API trong ProductController

Mở ProductController.java và thêm các API tương ứng để gọi các phương thức repository vừa định nghĩa:

```

// — 7. API Tìm kiếm sản phẩm theo từ khóa tên (Chương 5) —
@GetMapping("/search")
public List<Product> searchProducts(@RequestParam String keyword) {
    return productRepository.findByNameContainingIgnoreCase(keyword);
}

// — 8. API Lọc sản phẩm theo khoảng giá (Chương 5) —
@GetMapping("/price-range")
public List<Product> getProductsByPriceRange(
    @RequestParam BigDecimal min,
    @RequestParam BigDecimal max) {
    return productRepository.findByPriceBetween(min, max);
}

// — 9. API Lấy các sản phẩm đắt tiền hơn mức tối thiểu (Chương 5) —
@GetMapping("/expensive")
public List<Product> getExpensiveProducts(@RequestParam BigDecimal minPrice) {
    return productRepository.findExpensiveProducts(minPrice);
}

// — 10. API Lấy Top 3 sản phẩm đắt nhất (Chương 5 - Native SQL) —
@GetMapping("/top3")
public List<Product> getTop3Expensive() {
    return productRepository.findTop3ExpensiveProducts();
}

// — 11. API Cập nhật tăng giá sản phẩm theo tỷ lệ (Chương 5 - Modifying) —
@PutMapping("/{id}/increase-price")
public ResponseEntity<String> increaseProductPrice(
    @PathVariable Long id,
    @RequestParam BigDecimal rate) {
    int updatedRows = productRepository.updateProductPrice(id, rate);
    if (updatedRows > 0) {
        return ResponseEntity.ok("Cập nhật giá thành công cho sản phẩm ID: " +
id);
    }
    return ResponseEntity.status(HttpStatus.NOT_FOUND).body("Không tìm thấy sản
phẩm ID: " + id);
}
}

```

## Bước 3: Kiểm thử các API bằng cURL/Postman

- Tìm kiếm sản phẩm chứa chữ 'Dell': GET <http://localhost:8080/api/products/search?keyword=Dell>
- Tìm sản phẩm giá trong khoảng 30,000,000 đến 40,000,000:  
GET <http://localhost:8080/api/products/price-range?min=30000000&max=40000000>
- Cập nhật tăng giá sản phẩm ID 1 lên 10%: PUT <http://localhost:8080/api/products/1/increase-price?rate=1.1>

Quan sát Console log. Bạn sẽ thấy đối với API /expensive, Hibernate sinh ra câu SQL JPQL (làm việc trên thực thể và thuộc tính Java), còn đối với API /top3, Hibernate sinh ra chính xác câu lệnh Native SQL `SELECT * FROM products ORDER BY price DESC LIMIT 3` được chỉ định.

### Bài 3: Cấu hình các mối quan hệ bảng (JPA Relationships)

Mục tiêu: Thiết lập và ánh xạ các quan hệ Một-Một, Nhiều-Một, Một-Nhiều, Nhiều-Nhiều hai chiều hoàn chỉnh trong mã nguồn và CSDL.

#### Bước 1: Thiết lập quan hệ Một - Nhiều giữa Category và Product

Tạo Entity Category trong package `vn.edu.gdu.springjpalab.entity`:

```
package vn.edu.gdu.springjpalab.entity;

import jakarta.persistence.*;
import java.util.ArrayList;
import java.util.List;

@Entity
@Table(name = "categories")
public class Category {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(name = "category_name", nullable = false, unique = true, length = 100)
    private String name;

    @OneToMany(mappedBy = "category", cascade = CascadeType.ALL, orphanRemoval = true, fetch = FetchType.LAZY)
    private List<Product> products = new ArrayList<>();

    protected Category() {}

    public Category(String name) { this.name = name; }

    public Long getId() { return id; }
    public String getName() { return name; }
    public void setName(String name) { this.name = name; }
    public List<Product> getProducts() { return products; }
    public void setProducts(List<Product> products) { this.products = products; }
}
```

Cập nhật tệp `Product.java` để bổ sung trường khóa ngoại `category_id` kết nối tới `Category`:

```
// — Bổ sung thuộc tính quan hệ (Chương 5) —
@ManyToOne(fetch = FetchType.LAZY)
@JoinColumn(name = "category_id")
private Category category;

// Cập nhật constructor có tham số để thiết lập Category
public Product(String name, BigDecimal price, String sku, Category category) {
    this.name = name;
    this.price = price;
    this.sku = sku;
    this.category = category;
    this.createdAt = LocalDateTime.now();
}
```

```

    }

    public Category getCategory() { return category; }
    public void setCategory(Category category) { this.category = category; }

```

## Bước 2: Thiết lập quan hệ Một - Một giữa User và Profile

Tạo Entity Profile trong package vn.edu.gdu.springjpalab.entity:

```

package vn.edu.gdu.springjpalab.entity;

import jakarta.persistence.*;

@Entity
@Table(name = "profiles")
public class Profile {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(name = "bio", length = 500)
    private String bio;

    @Column(name = "avatar_url")
    private String avatarUrl;

    @OneToOne(mappedBy = "profile")
    private User user;

    protected Profile() {}

    public Profile(String bio, String avatarUrl) {
        this.bio = bio;
        this.avatarUrl = avatarUrl;
    }

    // Getters/Setters...
}

```

Tạo Entity User trong package vn.edu.gdu.springjpalab.entity:

```

package vn.edu.gdu.springjpalab.entity;

import jakarta.persistence.*;

@Entity
@Table(name = "users")
public class User {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(name = "username", nullable = false, unique = true, length = 50)
    private String username;

    @OneToOne(cascade = CascadeType.ALL, fetch = FetchType.LAZY)
    @JoinColumn(name = "profile_id", referencedColumnName = "id")

```

```

    private Profile profile;

    protected User() {}

    public User(String username, Profile profile) {
        this.username = username;
        this.profile = profile;
    }

    // Getters/Setters...
}

```

### Bước 3: Thiết lập quan hệ Nhiều - Nhiều giữa Student và Course

Tạo Entity Course trong package vn.edu.gdu.springjpalab.entity:

```

package vn.edu.gdu.springjpalab.entity;

import jakarta.persistence.*;
import java.util.HashSet;
import java.util.Set;

@Entity
@Table(name = "courses")
public class Course {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(name = "course_title", nullable = false, length = 150)
    private String title;

    @ManyToMany(mappedBy = "courses")
    private Set<Student> students = new HashSet<>();

    protected Course() {}

    public Course(String title) { this.title = title; }

    // Getters/Setters...
}

```

Cập nhật Entity Student.java (đã viết từ Chương 4) để định cấu hình quan hệ Nhiều-Nhiều:

```

// — Bổ sung thuộc tính quan hệ (Chương 5) —
@ManyToMany(cascade = {CascadeType.PERSIST, CascadeType.MERGE})
@JoinTable(
    name = "student_course", // Tên bảng liên kết trung gian tự sinh
    joinColumns = @JoinColumn(name = "student_id"), // Khóa ngoại trỏ về bảng
    inverseJoinColumns = @JoinColumn(name = "course_id") // Khóa ngoại trỏ về
    // hiện tại (students)
    // bảng đối diện (courses)
)
private java.util.Set<Course> courses = new java.util.HashSet<>();

// Hàm tiện ích hỗ trợ liên kết dữ liệu hai chiều nhanh chóng
public void enrollInCourse(Course course) {
    this.courses.add(course);
}

```

```
        course.getStudents().add(this);
    }

    public java.util.Set<Course> getCourses() { return courses; }
    public void setCourses(java.util.Set<Course> courses) { this.courses = courses; }
```

#### Bước 4: Khởi chạy và xác nhận cấu trúc cơ sở dữ liệu

Khởi động ứng dụng Spring Boot. Quan sát log sinh cấu trúc bảng tự động của Hibernate trên Console. Bạn sẽ thấy các lệnh DDL được thực thi (categories, products, profiles, users, students, courses, student\_course).

Đăng nhập vào MySQL Workbench để chạy lệnh kiểm tra:

```
USE gdu_jpa_lab;
SHOW TABLES;
DESCRIBE student_course;
```

## 5. Bài tập về nhà / Thực hành bổ sung

Sinh viên thực hiện mở rộng tính năng đăng ký học phần cho hệ thống:

1. Tạo các Repository interfaces: CategoryRepository, UserRepository, ProfileRepository, CourseRepository, và StudentRepository (nếu chưa có).
2. Viết REST API Endpoint POST /api/students/{studentId}/enroll/{courseId} trong một StudentController mới để tìm kiếm sinh viên và môn học dựa trên ID, gọi phương thức tiện ích student.enrollInCourse(course) và lưu lại thông tin sinh viên vào cơ sở dữ liệu.
3. Sử dụng Postman để tạo mới 2 Sinh viên, 2 Khóa học, thực hiện đăng ký và kiểm tra sự thay đổi dữ liệu trong bảng trung gian student\_course.

## 6. Câu hỏi ôn tập và củng cố kiến thức

1. JpaRepository kế thừa các interface nào? Hãy nêu sự khác biệt lớn nhất giữa findAll() của CrudRepository và findAll() của JpaRepository.
2. Hãy viết một câu SQL mô phỏng có mệnh đề LIMIT và OFFSET tương ứng khi client yêu cầu page = 2, size = 5 (lưu ý: số trang bắt đầu từ 0).
3. Lỗi N+1 query là gì? Tại sao việc thiết lập FetchType.EAGER cho các collection quan hệ @OneToMany lại là nguyên nhân chính dẫn đến lỗi này?
4. Trong quan hệ hai chiều, từ khóa mappedBy có tác dụng gì? Nếu quên khai báo mappedBy ở phía Inverse Side thì Hibernate sẽ xử lý như thế nào và gây ra hệ quả gì dưới Database?

## 7. Tài liệu tham khảo

1. Spring Data JPA Reference Documentation: <https://docs.spring.io/spring-data/jpa/reference/>

2. Baeldung Guides on Spring Data JPA: <https://www.baeldung.com/category/persistence/spring-persistence>
3. Vlad Mihalcea Blog (Hibernate Expert Performance Tips): <https://vladmihalcea.com/>